

Compliance Report

One thing I have learned throughout my career in network operations is that reliability and compliance are not achieved by a single process or a final approval step. They are built through consistent procedures, automation, monitoring, and having the right information available when decisions need to be made. After reading the DevOps Handbook case studies “Proving Compliance in Regulated Environments (2015)” and “Relying on Production Telemetry for ATM Systems (2013)”, I noticed both examples focused on a similar challenge regarding how an organization can maintain control and reliability while still improving their ability to deliver changes quickly. These case studies are involving software development environments, but I see similarities to challenges my network operations teams face regarding change management, risk reduction and using data to validate if a deployment was successful or if it needs to be rolled back or troubleshoot for failures.

The case study “Proving Compliance in Regulated Environments (2015)” focused on the challenges faced when operating in industries that require strict compliance controls. Compliance activities were often dependent on manual reviews, screenshots, documentation, approvals, and evidence collected after changes were completed. These processes were created to reduce risk, but they were time consuming and created delays between development teams and operations teams. It seems the main point was that we don’t have to choose between DevOps practices and compliance requirements, but compliance should be part of the deployment pipeline and use automation, traceability, and continuous validation to generate audit evidence automatically. Instead of manual audits, the shift is to provide auditors direct access to centralized logging and monitoring tools to validate controls and operation as needed.

From spending 20 years in Network Operations and this case study from 2015, I can totally relate to this previous change management processes being very manual and time consuming. These manual reviews were a necessity at the time to ensure the documentation and changes to be made were properly reviewed and approved to reduce risks. Fast forward to today where changes need to be implemented much quicker, it is critical to incorporate automation for validation checks and provide better visibility into the

workflow. The goal doesn't remove governance, but it makes governance more efficient and part of the process rather than a task at the end.

The second case study about "Relying on Production Telemetry for ATM Systems (2013)", focused on the importance of using real-time data from production environments to improve reliability and detect anomalies. This case study discussed how a developer embedded malicious software that put ATM machines in maintenance mode outside the normal schedule for the purpose of withdrawing cash. This malicious code was missed by the code review and approval process, but thanks to the Telemetry the unscheduled maintenance mode was detected early. This demonstrated that the telemetry can also be valuable for detecting security issues and unauthorized behavior.

The main lesson I took from this case study is that monitoring should not only be viewed as a tool for detecting failures. Production telemetry provides valuable feedback that can help teams understand whether changes are successful as well as identifying unexpected or suspicious behavior. This aligns closely with DevOps principles because continuous feedback allows teams to make smaller adjustments, identify system and security issues earlier, and improve reliability over time.

This concept aligns to my current experience in network operations where I have access to telemetry tools that provide real-time Key Performance Indicators (KPIs) that we utilize heavily to monitor the health of the network. These tools allow our engineers to determine whether network changes made achieve the expected result or whether additional troubleshooting or a rollback is needed. Like the ATM case study, having real-time visibility reduces the need to rely only on assumptions and allows our network and engineering teams to make decisions based on real time performance data. After reviewing both case studies, I noticed a common theme regarding compliance and monitoring no longer being viewed as separate responsibilities as they are both critical parts of protecting the deployment pipeline. When organizations integrate these practices into their normal workflows, they can improve security and reliability.

In my silo for the last 20 years, I've seen many improvements and just assumed my company had the right people in place that were listening to our feedback regarding much-needed improvements. Going through this course and case studies like these two have shown me that my team's previous frustrations and the company's improvements were not isolated to my organization, but rather collectively across many DevOps industries. We all face many of the same issues and those great principles are being implemented which is making a real difference! DevOps is not about removing controls or reducing accountability, it is about improving the way organizations manage risk through automation, collaboration, and continuous feedback.